

Spring Framework & Microservices Architecture – Architect Interview Questions and Answers

Q: What problem does Spring Framework solve?

A: Spring reduces tight coupling through Dependency Injection (DI) and provides a comprehensive infrastructure for building enterprise Java applications. It simplifies configuration, improves testability, and promotes clean architecture.

Q: Explain Dependency Injection in Spring.

A: DI is a design principle where object dependencies are provided externally rather than created by the object itself. Spring supports constructor, setter, and field injection, improving modularity and testability.

Q: Difference between BeanFactory and ApplicationContext?

A: ApplicationContext is a superset of BeanFactory. It supports internationalization, event propagation, AOP, and eager bean initialization, making it suitable for enterprise applications.

Q: What is Spring Boot and why is it used?

A: Spring Boot simplifies Spring application development by providing auto-configuration, embedded servers, and opinionated defaults, enabling rapid microservices development.

Q: How does Spring Boot auto-configuration work?

A: It uses classpath scanning, conditional annotations, and starter dependencies to configure beans automatically based on available libraries and properties.

Q: What are starters in Spring Boot?

A: Starters are curated dependency descriptors that simplify dependency management by bundling commonly used libraries for specific functionalities.

Q: Explain Spring Profiles.

A: Profiles allow environment-specific configurations (dev, test, prod). Beans and properties can be activated conditionally based on the active profile.

Q: What is AOP and where is it used?

A: Aspect-Oriented Programming separates cross-cutting concerns like logging, security, and transactions using aspects, pointcuts, and advice.

Q: How does transaction management work in Spring?

A: Spring provides declarative and programmatic transaction management, commonly using @Transactional, abstracting underlying transaction APIs.

Q: What is a microservices architecture?

A: It is an architectural style where applications are composed of small, independently deployable services that communicate over lightweight protocols.

Q: Benefits of microservices over monolith?

A: Independent scaling, faster deployments, technology diversity, fault isolation, and better alignment with agile teams.

Q: What are the challenges of microservices?

A: Increased complexity, distributed transactions, network latency, data consistency, monitoring, and operational overhead.

Q: What is Spring Cloud?

A: Spring Cloud provides tools for building distributed systems, including service discovery, configuration management, circuit breakers, and API gateways.

Q: Explain Service Discovery.

A: Service discovery allows services to dynamically find each other. Tools like Eureka or Consul maintain a registry of service instances.

Q: What is an API Gateway?

A: An API Gateway is a single entry point that handles routing, authentication, rate limiting, and aggregation for microservices.

Q: How do you handle configuration in microservices?

A: Using centralized configuration servers (e.g., Spring Cloud Config) with externalized configuration stored in Git or Vault.

Q: What is Circuit Breaker pattern?

A: It prevents cascading failures by stopping calls to failing services. Resilience4j is commonly used with Spring Boot.

Q: Explain Saga pattern.

A: Saga manages distributed transactions using a sequence of local transactions coordinated via events or orchestration.

Q: How do you secure microservices?

A: Using OAuth2, OpenID Connect, JWT, API gateways, and centralized authentication servers like Keycloak.

Q: What is eventual consistency?

A: A consistency model where systems become consistent over time, commonly used in distributed microservices architectures.

Q: How do you implement observability?

A: Using centralized logging (ELK), metrics (Prometheus), and distributed tracing (Zipkin, Jaeger).

Q: How do you scale microservices?

A: Horizontally using container orchestration platforms like Kubernetes with auto-scaling policies.

Q: What is Kubernetes role in microservices?

A: Kubernetes manages containerized applications, handling deployment, scaling, service discovery, and self-healing.

Q: How do you design a resilient microservice system?

A: By applying patterns like bulkhead, retry, timeout, circuit breaker, and graceful degradation.

Q: Monolithic vs Microservices – when to choose which?

A: Monoliths suit small teams and simple domains. Microservices fit large, complex systems requiring scalability and independent deployments.

Preparation Tip

For architect-level interviews, focus on trade-offs, real-world experience, failure scenarios, and system design discussions rather than only definitions.